CSCI3100 Software Engineering

Software Requirements Specification v1.2

Author(s):

Archie Hamilton - [1155256512]

Chun Wang YIP - [1155192968]

Hou Fong Chan - [1155200045]

Member(s):

Luke Carroll - [1155255972]

Faculty of Engineering

Department of Computer Science and Engineering

The Chinese University of Hong Kong

TaskFlow

1. Introduction

This SRS specifies the requirements for TaskFlow, a Jira-inspired web application for team task scheduling, file organization, and personal brainstorming via a canvas. It is intended to guide implementation, testing, and course assessment.

1.1 Scope

TaskFlow provides: (1) invite-only team access via activation keys; (2) owner/admin/member roles; (3) tasks with calendar/list/completion views; (4) activity dashboard; (5) files/folders with visibility controls; (6) shared departments; and (7) a personal canvas board per user.

1.2 Document Convention

Account/User: A registered identity authenticated via email OTP or optional GitHub OAuth.

Keywords: Shall = mandatory, Should = recommended, May = optional.

Requirement identifiers:

- Functional: `FR-<domain>-<number>` (e.g., `FR-TASK-2`)
- Non-functional: `NFR-<category>-<number>` (e.g., `NFR-SEC-1`)
- Business rules: `BR-<number>`

*FR/NFR presentation format follows the project's traceability-style ("TRACEABILITY.md") structure: Requirement ID → Description → Implementation → Tests → Status.

1.3 Intended audience and reading suggestions

- Audience: product stakeholders, developers, testers, course staff
- Reading:
  - Overview: Sections 1-2
  - Feature requirements: Section 3
  - Interfaces/constraints: Section 4
  - Quality requirements: Section 5
  - Terms/models/TBD: Appendices

1.4 Reference

- Software Requirements Specification template repository provided by the course.
- Software Requirements Specification template repository provided by the course.
- Software Engineering Lecture Notes (course materials).
- Sommerville, I. Software Engineering (7th Edition) (reference book used by the course).
- Atlassian Jira product description (reference product for feature framing).

2. Overall Description

2.1 Product Overview

TaskFlow is a multi-team web application. Each team has isolated data (tasks, files, folders, departments). Users may belong to multiple teams and select an active team context.

2.2 Product Functions (High-Level)

- Authentication: email OTP login/signup; optional GitHub OAuth login; logout
- Licence/team gating: activation key creates/authorizes teams and admin features
- Team administration: owner/admin manage membership; owner deletes team
- Tasks: create/edit/assign/status changes; personal and shared task rules
- Views: calendar view, list view, completion view; saved view preferences
- Dashboard: activity feed + tasks due today; quick status updates
- Files: folder tree + list/grid; upload/preview/download; visibility controls
- Canvas: personal board with nodes/connectors; undo/redo; autosave; export

2.3 User Classes

Owner: Creates the team during initial activation, sees activation keys and usage, can delete the team, manages admins and members.

Admin: Manages members and shared team resources; can assign tasks to any team member and upload admin-only files; cannot delete the team.

Member: Accesses team after invitation; can create personal tasks and manage tasks assigned to them; cannot access admin dashboard or admin-only files.

2.4 Key Constraints

- Users must be enrolled in a team (invited) or must activate a valid key before accessing core features.
- Activation key is formatted in AAAA-BBBB-CCCC (uppercase alphanumeric grouped by 4).
- File upload size limit is 50 MB per file.
- Role-based visibility rules apply to tasks, files, and folders.
- Canvas is personal-only; it is not shared across team members in the current scope.
- UI responsiveness: TaskFlow is desktop-first; mobile/tablet responsiveness is out of scope for this release (small-screen layouts may overflow).

2.5 Assumptions/Dependencies

- Email delivery service available (SMTP/Mailpit/dev SMTP) for OTP.
- GitHub OAuth requires valid client credentials (if enabled).
- Database connectivity available (MongoDB).
- Team membership is mandatory to access core features (invite or activation required).

3. External Interface Requirements

3.1 UI Summary

Navigation: Dashboard, Calendar, Files, Canvas. Top bar provides team context and Admin Dashboard for owners/admins. A bottom-left Admin Access Panel accepts activation keys.

3.2 Software Interfaces

MongoDB: Primary persistence layer for users, teams, tasks, departments, files metadata, folders, and canvas boards.

Email Delivery: Email delivery service (SMTP/Mailpit/dev SMTP) for OTP

GitHub OAuth: Optional external identity provider for login when configured.

3.3 Communications

Frontend and backend communicate over HTTP(S) using JSON APIs; files use streamed download responses.

4. Requirements

Requirement status legend:

- Implemented: present and supported in current release
- Partial: present but with known limitation / incomplete verification coverage
- Planned: specified for completeness but not required for Release 1.3

4.1 Functional Requirements

4.1.1 Authentication, Activation, and Teams

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-UM-1	User shall initiate login by entering a valid email and requesting OTP. Fit: invalid email rejected with clear message.	backend auth controller; frontend login page	backend auth tests; frontend login tests	Implemented
FR-UM-2	System shall generate OTP and send to provided email. Fit: OTP generated per	backend auth + email services	backend auth tests	Implemented

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
	request; deliver attempt logged.			
FR-UM-3	System shall verify OTP and create authenticated session. Fit: valid OTP within expiry yields active session.	backend auth controller/service	backend auth tests	Implemented
FR-UM-4	System shall reject incorrect/expired OTP. Fit: error code + non-200 response; no session issued.	backend auth service	backend auth tests	Implemented
FR-UM-5	User shall be able to logout; system shall terminate session.	backend auth controller	backend auth tests	Implemented
FR-UM-6	System shall auto-create account on first successful OTP login. Fit: new user record created once.	backend auth service + user model	backend auth tests	Implemented
FR-OAUTH-1	System shall support GitHub OAuth login as an alternative authentication method. Fit: successful callback logs user in and redirects to app home.	backend GitHub OAuth service/controller; frontend login page	backend github-oauth tests	Implemented
FR-OAUTH-2	System shall fail gracefully when GitHub OAuth is not configured. Fit: user redirected to login with actionable error.	backend GitHub OAuth controller	backend github-oauth tests	Implemented
FR-LIC-1	System shall allow entering activation key in UI panel. Fit: key accepted only if matches allowed pattern; errors shown otherwise.	admin controller; admin panel UI	backend admin tests; frontend admin panel tests; e2e smoke	Implemented

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-LIC-2	System shall validate activation key scheme. Fit: invalid key rejected; valid key links to licence/team record; usage decremented.	adminKey util; licence key model	backend admin tests	Implemented
FR-LIC-3	System shall unlock admin-only features after activation. Fit: UI elements appear only for authorized roles; APIs enforce same.	top bar; admin panel	e2e smoke	Implemented
FR-LIC-4	System shall display activation status in UI. Fit: user can see current activation state; owners see key/usage where allowed.	admin panel UI	frontend admin panel tests	Implemented
FR-LIC-5	System shall enforce time-limited activation keys. Fit: expired key rejected; rotation supported.	licence key model + adminKey	backend admin tests	Implemented
FR-LIC-6	System should accept both 12-char and 16-char licence formats to match course format guidance. Fit: both patterns pass validation; storage normalized.	adminKey util	admin tests	Planned (recommended to ensure format alignment)
FR-TEAM-1	Team membership shall be invite-only (or via key activation). Fit: uninvited users cannot access team-scoped modules.	teams controller	teams tests	Implemented
FR-TEAM-2	Access control shall be enforced per team context. Fit: cross-team	team middleware	teams/tasks/files tests	Implemented

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
	data access rejected (403/404).			
FR-TEAM-3	Owner shall be able to delete a team with confirmation. Fit: deletion requires explicit confirmation step.	teams controller; top bar UI	teams tests	Implemented
FR-TEAM-4	Owner shall complete team setup (naming) before other accounts can finalize access. Fit: “pending setup” blocks secondary activation completion until owner finishes.	teams controller; app layout	admin tests; e2e smoke	Implemented
FR-TEAM-5	Owner setup shall create team and bind key to that team. Fit: key-team link created once; repeat setup rejected.	teams controller; app layout	teams + admin tests	Implemented
FR-TEAM-6	Admins shall be able to add members; only owners shall be able to add admins. Fit: role enforcement verified server-side.	teams controller + RBAC	teams tests	Implemented

4.1.2 Dashboard, Tasks, and Views

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-DASH-1	System shall provide team dashboard for task and file activity.	dashboard page	dashboard UI tests	Implemented
FR-DASH-2	Dashboard shall provide activity feed + due-today list.	dashboard page	dashboard UI tests	Implemented
FR-DASH-3	Dashboard shall allow task status change from feed/details.	dashboard page	UI checks	Implemented

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-DASH-4	Users shall search activity feed entries. Fit: search filters by keyword and respects visibility rules.	dashboard page	UI checks	Implemented
FR-TASK-1	System shall allow creating tasks with required attributes. Fit: missing required fields rejected.	tasks controller; calendar page	tasks tests; calendar UI tests	Implemented
FR-TASK-2	System shall support assigning one or more collaborators. Fit: multi-assignee stored and retrieved correctly.	tasks controller; multi-select UI	tasks tests	Implemented
FR-TASK-3	Authorized users shall update task attributes. Fit: only creator (or admin/owner for shared tasks) can edit; unauthorized edit rejected.	tasks controller	tasks tests	Implemented
FR-TASK-4	Assignees shall mark task completed via status change. Fit: status transitions persist; visible in completion view.	tasks controller	tasks tests	Implemented
FR-TASK-5	Users shall delete tasks with confirmation. Fit: UI confirmation required; deletion irreversible.	tasks controller; calendar page	tasks tests	Implemented
FR-TASK-6	Users shall filter tasks by priority and department. Fit: filters change visible set deterministically.	calendar page	UI checks	Implemented
FR-TASK-7	System shall enforce personal task privacy: personal tasks are visible only to creator. Fit: API never returns personal tasks to others.	tasks controller + access rules	tasks tests	Implemented
FR-VIEW-1	System shall provide list view of tasks.	calendar page	calendar UI tests	Implemented
FR-VIEW-2	Users shall change task status directly from a task view (without full edit).	calendar page	UI checks	Implemented
FR-VIEW-3	System shall provide calendar view (month navigation + day list).	calendar page	calendar UI tests	Implemented

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-VIEW-4	System shall remember last view selection. Fit: restore on revisit/refresh.	calendar page	UI checks	Implemented
FR-VIEW-5	System shall provide completion view (list of completed tasks with revert).	calendar page	UI checks	Implemented

4.1.3 Files, Folders, Departments, and Canvas

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-ATT-1	Users shall upload files (≤ 50MB). Fit: oversize rejected with clear message.	files controller; files page	files tests	Implemented
FR-ATT-2	System shall store file metadata (name, size, type, department, visibility, owner/team).	file model	files tests	Implemented
FR-ATT-3	System shall restrict file access by visibility rules. Fit: unauthorized access returns 403/404.	files controller	files tests	Implemented
FR-ATT-4	System shall handle upload errors and enforce size limits.	files controller	manual UI checks	Partial
FR-ATT-5	Upload/download shall require authenticated membership.	auth middleware	files tests	Implemented
FR-FOLD-1	Users shall create folders (depth-limited). Fit: invalid depth rejected.	folders controller; files page	folders tests	Implemented
FR-FOLD-2	Users shall list folders in a directory tree view. Fit: inaccessible folders hidden per rules.	folders controller; files UI	folders tests; UI checks	Implemented
FR-FOLD-3	Users shall delete folders with confirmation and deletion count. Fit: UI shows impact; deletion removes contained items.	folders controller; files UI	folders tests; UI checks	Implemented

Requirement ID	Description (incl. fit criteria)	Implementation (key files)	Tests	Status
FR-FILE-1	Users shall preview supported files in-browser.	files UI + backend preview endpoint	UI checks	Implemented
FR-FILE-2	Users shall download files they are authorized to access.	files controller	files tests	Implemented
FR-DEPT-1	Owner/admin shall create and delete departments shared within team. Members cannot.	departments controller; files/calendar UI	departments tests	Implemented
FR-DEPT-2	Each task/file shall belong to exactly one department.	tasks/files models + validation	tasks/files tests	Implemented
FR-CAN-1	System shall provide personal Canvas board per user with autosave.	canvas page; canvas controller	canvas tests; canvas UI tests	Implemented
FR-CAN-2	Canvas shall support connectors between nodes (line/arrow).	canvas page	manual UI checks	Implemented
FR-CAN-3	Canvas shall export board as PNG/PDF.	canvas page	manual UI checks	Implemented
FR-CAN-4	Canvas shall support undo/redo and snap-to-grid toggle.	canvas page	manual UI checks	Implemented

4.2 Non-Functional Requirements

4.2.1 NFR List

Requirement ID	Description (incl. fit criteria)	Implementation	Tests	Status
NFR-SEC-1	Auth shall use OTP and secure sessions. Fit: session required for all protected endpoints.	auth service + auth middleware	auth tests	Implemented
NFR-SEC-2	OTP shall expire and enforce retry limits.	auth service + env config	auth tests	Implemented
NFR-SEC-4	Access control shall be enforced per team.	auth/team middleware	teams tests	Implemented
NFR-SEC-5	If HTTPS is available, client–server communication should use HTTPS.	deployment config	N/A	Partial

Requirement ID	Description (incl. fit criteria)	Implementation	Tests	Status
NFR-PERF-1	For typical team usage (≤50 users/team), primary screens (Dashboard/Calendar/Files) should load within 2 seconds on campus Wi-Fi.	caching + efficient queries	manual timing checks	Partial
NFR-QUAL-3	Codebase shall maintain clear separation for maintainability.	monorepo structure	repo review	Implemented
NFR-QUAL-4	System should handle unexpected errors gracefully (no raw stack traces to users).	API error handler + UI messages	auth tests	Partial
NFR-QUAL-5	System should be portable across OS (Linux/Windows).	Node + web stack	N/A	Partial
NFR-UX-1	A first-time invited user should be able to find Calendar, create a personal task, and mark it complete without external help.	UI copy + affordances	usability walkthrough	Partial
NFR-UX-2	Common operations (create task, change status) should take ≤3 user actions from the relevant view.	calendar/dashboard interactions	UI review	Implemented

5. Other Requirements

5.1 Business Rules (BR)

- BR-1: Only authenticated users may access any team data.
- BR-2: Users must belong to a team (via invite or key activation) to access core modules.
- BR-3: Only owner/admin can assign tasks to others; members can only self-assign.
- BR-4: Personal tasks are private to creator regardless of role.
- BR-5: Members cannot see admin-only files; cannot create/delete departments.
- BR-6: A task/file belongs to exactly one department.

5.2 Acknowledgement

The authors employed ChatGPT 5.2 to assist in the appendix section, refining prose, enhancing clarity, and ensuring consistent formatting throughout this document. The core ideas, structure, and cited sources originate from the authors' knowledge, and research.

6. Verification and Acceptance

Verification uses automated backend tests, selected frontend tests, end-to-end smoke tests, and manual UI verification for highly interactive features (canvas, previews, multi-browser behavior).

6.1 Acceptance Criteria

The system is accepted when all requirements marked Implemented are satisfied in the deployed build and CI tests pass.

Appendix A. Glossary

Activation Key / Licence Key: key used to authorize team/admin access (TaskFlow uses `AAAA-BBBB-CCCC` pattern; optionally accept `AAAA-BBBB-CCCC-DDDD` for course alignment).

Account/User: A registered identity authenticated via email OTP or optional GitHub OAuth.

Team: A collaboration scope. Data (tasks, files, departments) is isolated per team.

Owner: The first user who redeems an activation key for a team and completes team setup.

Admin: A user with elevated team permissions (inviting members, assigning team tasks, uploading admin-only files).

Member: A standard user who can access the team after invitation; can create personal tasks and view assigned tasks.

Activation Key: A licence-style key used to activate/admin-enable a team, formatted as AAAA-BBBB-CCCC.

Personal Task: A task created by a member and self-assigned; visible only to the creator.

Admin-only File: A file visible only to team owner and admins.

Appendix B. Analysis Models (textual)

Primary use cases

UC-1: OTP login and session creation

UC-2: GitHub OAuth login (optional)

UC-3: Activate key → become owner/admin → complete team setup

UC-4: Owner/admin invite member → member gains access

UC-5: Create task (personal or shared), change status, filter/view tasks

UC-6: Upload file, set visibility, preview/download, folder organization

UC-7: Use Canvas personal board, autosave, export

High-level architecture overview (text)

- Client (React) calls REST/JSON backend (Node/Express).
- Backend enforces auth + team middleware; stores/retrieves from MongoDB.
- Email service used for OTP delivery; OAuth used for GitHub login.

Appendix C. To-Be-Determined List

- Optional sharing of Canvas boards between team members (currently personal-only).
- Richer file preview support for additional formats (e.g., Office documents).
- Enhanced performance benchmarking and load testing thresholds.
- Alternative activation key formats and a user-facing key-file import workflow if required by future course clarification.

Appendix D. GitHub Link & Revision History

GitHub Repo: https://github.com/lukecarroll15/CSCI3100-Project

Current version: v1.2

Document status: Final

Version	Date (DD/MM/YYYY)	Author(s)	Description of Changes
v0.8	17/11/2025	Hou Fong CHAN; Chun Wang YIP	Initial draft completed (baseline requirements captured).
v1.0	26/12/2025	Chun Wang YIP	Finalized content revision; requirements completed and consolidated for submission.
v1.1	27/12/2025	Archie Hamilton	Formatting and layout pass to ensure document consistency and submission readiness.
v1.2	28/12/2025	Chun Wang YIP	Final revision and formatting; final quality checks and polishing for submission.